PYTHON ACTIVITY-BASED LEARNING

10 Selected Case Studies – Basic Python Programs

Prepared using the case studies listed in the supplied Python Activity-Based Learning PDF.

Note: Flowcharts are shown in a simple text-arrow format. Output is provided as text under test cases; screenshots are not included.

1. Attendance Recovery Computation

Problem statement: Calculate a student's current attendance percentage and find the minimum number of consecutive classes the student must attend to reach a required attendance percentage.

Objectives

- Calculate attendance percentage.
- Find the minimum future classes needed to reach the target.

Assumptions

- Classes attended and total classes are non-negative integers.
- The student attends every future class.
- The target attendance is 75%.

Algorithm

1. Start.
2. Read attended classes and total classes.
3. Calculate current attendance percentage.
4. If total classes is zero, display an error.
5. Otherwise, increase future classes one by one until attendance reaches 75%.
6. Display current percentage and required classes.
7. Stop.

Flowchart: Start → Input attended and total classes → Validate total classes → Calculate percentage → Check target using a loop → Display result → Stop

Pseudocode

```
BEGIN
  INPUT attended, total
  IF total <= 0 THEN print error
  ELSE
    percentage = attended / total * 100
    future = 0
    WHILE (attended + future) / (total + future) * 100 < 75
      future = future + 1
    END WHILE
    PRINT percentage, future
  END IF
END
```

Python source code

```
attended = int(input("Enter classes attended: "))
total = int(input("Enter total classes: "))

if attended < 0 or total <= 0 or attended > total:
    print("Invalid input")
else:
    percentage = (attended / total) * 100
    print("Attendance percentage:", percentage, "%")
```

```
future = 0

while percentage < 75:
    attended = attended + 1
    total = total + 1
    future = future + 1
    percentage = (attended / total) * 100

print("Classes needed:", future)
```

Test cases and expected output

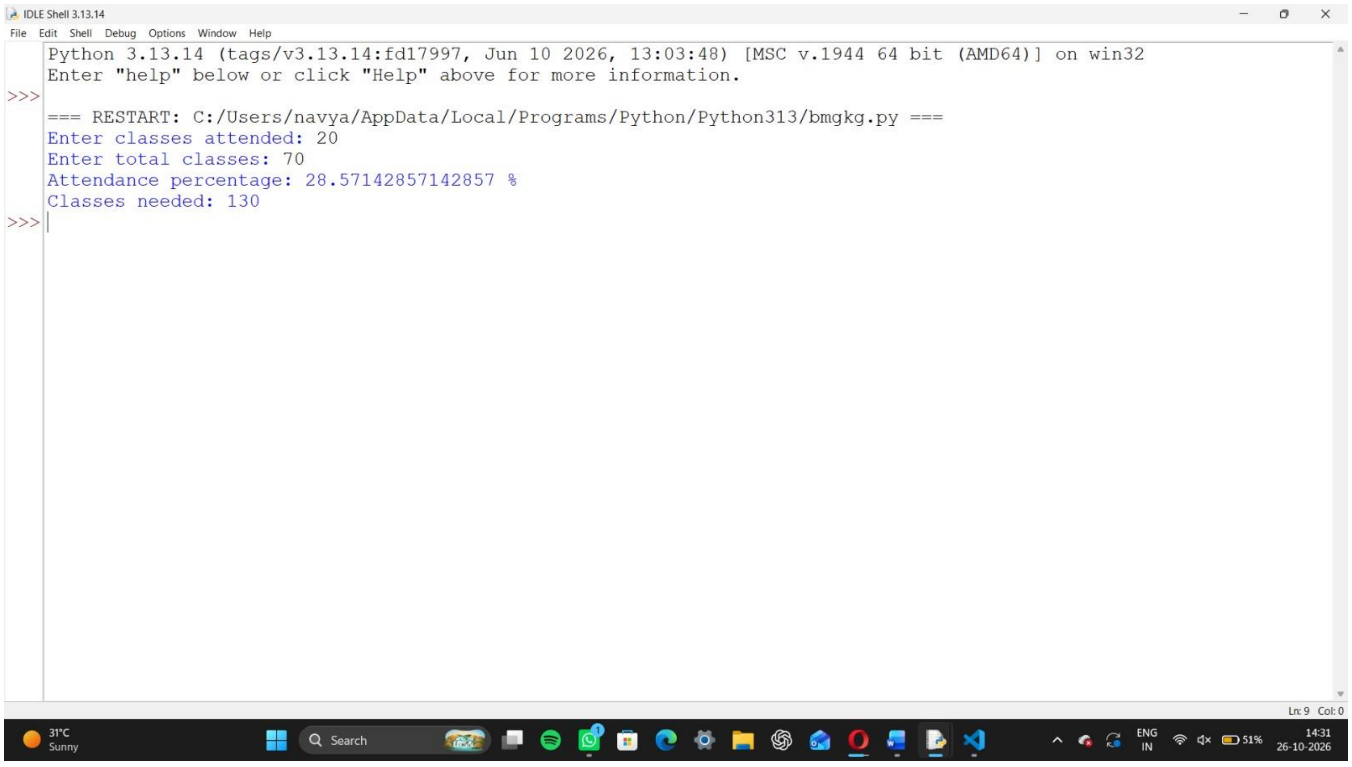
Test case 1: Input: 30, 40

Expected output: Current attendance: 75.0 % | Classes needed to reach 75%: 0

Test case 2: Input: 20, 40

Expected output: Current attendance: 50.0 % | Classes needed to reach 75%: 40

Conclusion



2. ATM Transaction Sequence Simulation

Problem statement: Process deposits and withdrawals, update an account balance, and reject withdrawals that are greater than the available balance.

Objectives

- Update balance after each transaction.
- Prevent withdrawals when funds are insufficient.

Assumptions

- Opening balance is ₹1000.
- Transaction type is D for deposit or W for withdrawal.
- Only positive transaction amounts are accepted.

Algorithm

1. Start with balance ₹1000.
2. Read number of transactions.
3. For each transaction, read type and amount.
4. Deposit positive amounts or withdraw if balance is enough.
5. Otherwise show an invalid transaction message.
6. Display final balance.
7. Stop.

Flowchart: Start → Set opening balance → Read transactions → Check type and amount → Update or reject transaction → Repeat → Show final balance → Stop

Pseudocode

```
BEGIN
  balance = 1000
  INPUT n
  REPEAT n times
    INPUT type, amount
    IF amount <= 0 THEN print invalid
    ELSE IF type = D THEN balance = balance + amount
    ELSE IF type = W AND amount <= balance THEN balance = balance - amount
    ELSE print rejected
  PRINT balance
END
```

Python source code

```
balance = 1000
n = int(input("Number of transactions: "))

for i in range(n):
    kind = input("Enter D for deposit or W for withdrawal: ").upper()
    amount = float(input("Amount: "))
    if amount <= 0:
        print("Invalid amount")
    elif kind == "D":
        balance += amount
        print("Deposit successful")
    elif kind == "W" and amount <= balance:
        balance -= amount
        print("Withdrawal successful")
    elif kind == "W":
        print("Insufficient balance")
```

```
        else:
            print("Invalid transaction type")

    print("Final balance: Rs.", round(balance, 2))
```

Test cases and expected output

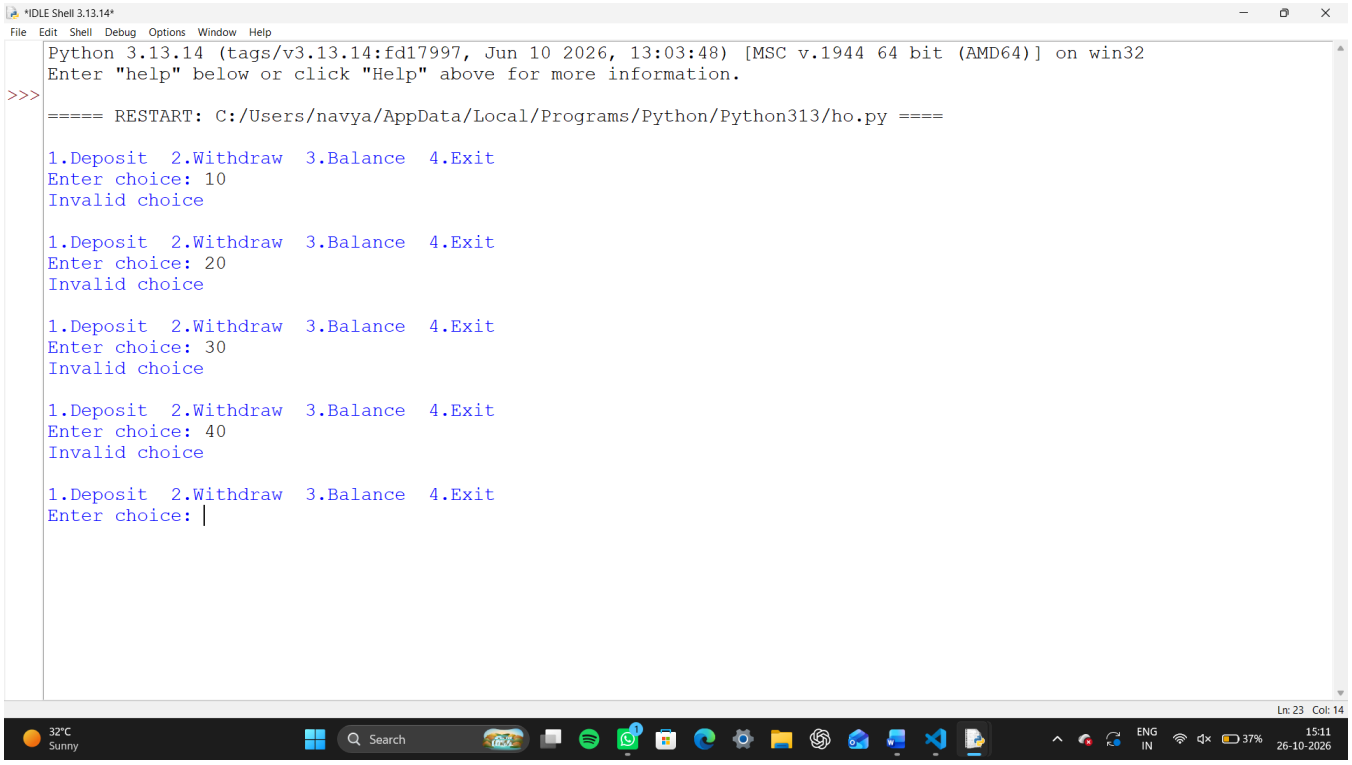
Test case 1: Input: 2, D, 500, W, 200,

Expected output: Deposit successful | Withdrawal successful | Final balance: Rs. 1300.0

Test case 2: Input: 1, W, 1500,

Expected output: Insufficient balance | Final balance: Rs. 1000

Conclusion



3. Warehouse Inventory Reconciliation

Problem statement: Compare opening stock, received stock, and dispatched stock for products to calculate the expected closing stock.

Objectives

- Calculate closing stock.
- Identify products with low or negative stock.

Assumptions

- All quantities are whole numbers.
- Expected closing stock = opening stock + received − dispatched.
- A closing stock below 5 is considered low.

Algorithm

1. Start.
2. Store product stock details in a list.
3. For each product, calculate closing stock.
4. Display closing stock.
5. If closing stock is below 5, display low-stock warning.
6. Stop.

Flowchart: Start → Read product stock list → Calculate closing stock for each product → Check low stock → Display results → Stop

Pseudocode

```
BEGIN
  FOR each product in stock list
    closing = opening + received - dispatched
    PRINT product, closing
    IF closing < 5 THEN PRINT low stock
  END FOR
END
```

Python source code

```
stock = [
    ("Pen", 20, 10, 18),
    ("Book", 15, 5, 8),
    ("Pencil", 12, 2, 11)
]

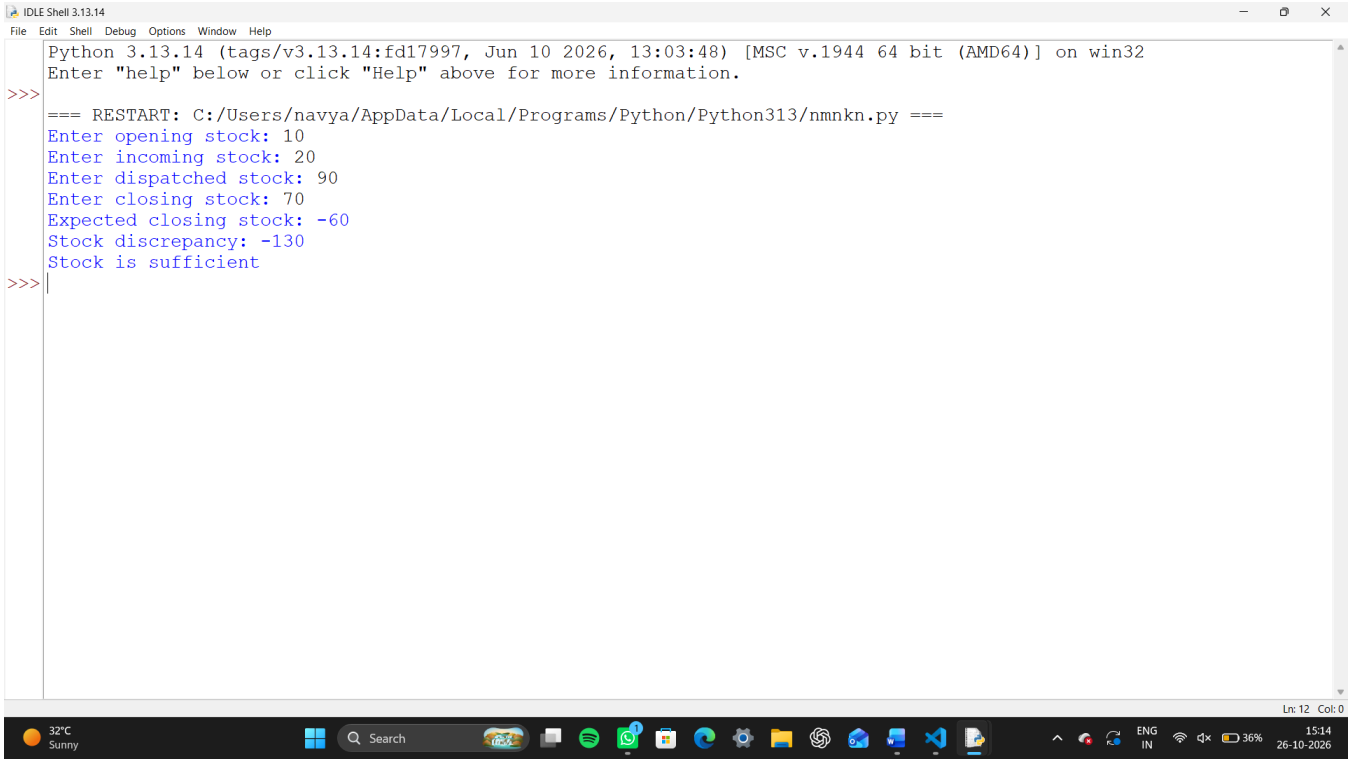
for product, opening, received, dispatched in stock:
    closing = opening + received - dispatched
    print(product, "closing stock:", closing)
    if closing < 5:
        print("Low stock warning")
```

Test cases and expected output

Test case 1: Input: Predefined sample data

Expected output: Pen closing stock: 12 | Book closing stock: 12 | Pencil closing stock: 3 | Low stock warning

Conclusion


IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/navya/AppData/Local/Programs/Python/Python313/nmnkn.py ===
Enter opening stock: 10
Enter incoming stock: 20
Enter dispatched stock: 90
Enter closing stock: 70
Expected closing stock: -60
Stock discrepancy: -130
Stock is sufficient
>>>
Ln: 12 Col: 0
32°C
Sunny
Search
ENG
IN
36%
15:14
26-10-2026

4. Student Performance Trend Analysis

Problem statement: Compare a student's marks in two tests for different subjects and identify whether marks improved or decreased.

Objectives

- Compare marks across two tests.
- Display improvement or decline for each subject.

Assumptions

- Marks are out of 100.
- The same subjects are used in both tests.
- A difference of zero means no change.

Algorithm

1. Start.
2. Store subject names and two test marks.
3. For each subject, calculate second-test marks minus first-test marks.
4. Display improved, decreased, or unchanged.
5. Stop.

Flowchart: Start → Store subject marks → Calculate difference → Check positive/negative/zero → Display trend → Stop

Pseudocode

```
BEGIN
  FOR each subject
    change = test2 - test1
    IF change > 0 print Improved
    ELSE IF change < 0 print Decreased
    ELSE print No change
  END FOR
END
```

Python source code

```
marks = [
    ("Python", 70, 82),
    ("Maths", 85, 78),
    ("English", 75, 75)
]

for subject, test1, test2 in marks:
    change = test2 - test1
    if change > 0:
        print(subject, "improved by", change, "marks")
    elif change < 0:
        print(subject, "decreased by", abs(change), "marks")
    else:
        print(subject, "no change")
```

Test cases and expected output

Test case 1: Input: Predefined sample data

Expected output: Python improved by 12 marks | Maths decreased by 7 marks | English no change

Conclusion

5. Fuel Consumption and Route Comparison

Problem statement: Calculate fuel required and travel cost for different vehicles, then identify the vehicle with the lowest journey cost.

Objectives

- Calculate fuel required using distance and mileage.
- Compare journey costs.

Assumptions

- Journey distance is 200 km.
- Fuel price is ₹105 per litre.
- Mileage values are greater than zero.

Algorithm

1. Start.
2. Set journey distance and fuel price.
3. Store vehicles and their mileage.
4. For each vehicle, calculate fuel needed and cost.
5. Display each cost.
6. Find and display the lowest-cost vehicle.
7. Stop.

Flowchart: Start → Set distance and fuel price → Read vehicle mileage list → Calculate fuel and cost → Compare costs → Display cheapest vehicle → Stop

Pseudocode

```
BEGIN
  distance = 200; price = 105
  FOR each vehicle
    fuel = distance / mileage
    cost = fuel * price
  FIND minimum cost
  PRINT all costs and cheapest vehicle
END
```

Python source code

```
distance = 200
fuel_price = 105
vehicles = [("Bike", 40), ("Car", 15), ("Scooter", 45)]
costs = []

for name, mileage in vehicles:
    fuel = distance / mileage
    cost = fuel * fuel_price
    costs.append((name, cost))
    print(name, "travel cost: Rs.", round(cost, 2))

cheapest = min(costs, key=lambda item: item[1])
print("Lowest-cost vehicle:", cheapest[0])
```

Test cases and expected output

Test case 1: Input: Predefined sample data

Expected output: Bike travel cost: Rs. 525.0 | Car travel cost: Rs. 1400.0 | Scooter travel cost: Rs. 466.67 | Lowest-cost vehicle: Scooter

Conclusion

This program solves the given problem using basic Python concepts such as variables, conditions, loops, strings, lists, nested lists, or tuples. It displays the required result for the sample data.

6. Repeated Transaction Detection

Problem statement: Check a list of transaction IDs and identify any IDs that appear more than once.

Objectives

- Find duplicate transaction IDs.
- Display each repeated ID only once.

Assumptions

- Transaction IDs are strings.
- The list is already available in the program.
- A repeated ID is treated as a possible duplicate.

Algorithm

1. Start.
2. Store transaction IDs in a list.
3. Check each ID against the seen list.
4. If already seen and not reported, add it to duplicates.
5. Otherwise add it to seen.
6. Display duplicates.
7. Stop.

Flowchart: Start → Read transaction IDs → Check each ID → Seen before? → Add to duplicate list or seen list → Display duplicates → Stop

Pseudocode

```
BEGIN
  seen = []; duplicates = []
  FOR each id in transactions
    IF id in seen AND id not in duplicates THEN add id to duplicates
    ELSE add id to seen
  PRINT duplicates
END
```

Python source code

```
transactions = ["T101", "T102", "T103", "T102", "T104", "T101"]
seen = []
duplicates = []

for tid in transactions:
    if tid in seen:
        if tid not in duplicates:
            duplicates.append(tid)
    else:
        seen.append(tid)

print("Repeated transaction IDs:", duplicates)
```

Test cases and expected output

Test case 1: Input: Predefined sample data

Expected output: Repeated transaction IDs: ['T102', 'T101']

Conclusion

This program solves the given problem using basic Python concepts such as variables, conditions, loops, strings, lists, nested lists, or tuples. It displays the required result for the sample data.

7. Password Policy Validation

Problem statement: Check whether a sample password has at least eight characters and contains both a letter and a digit.

Objectives

- Validate a password using simple string operations.
- Display whether it meets the given rules.

Assumptions

- Password length must be at least 8.
- Password must contain at least one letter and one digit.
- This is a classroom demonstration, not a complete security checker.

Algorithm

1. Start.
2. Read password.
3. Check its length.
4. Check whether it has a letter and a digit.
5. Display valid if all conditions are met; otherwise display invalid.
6. Stop.

Flowchart: Start → Input password → Check length → Check letter and digit → All conditions met? → Display result → Stop

Pseudocode

```
BEGIN
  INPUT password
  has_letter = any character is a letter
  has_digit = any character is a digit
  IF length >= 8 AND has_letter AND has_digit THEN print valid
  ELSE print invalid
END
```

Python source code

```
password = input("Enter sample password: ")
has_letter = any(ch.isalpha() for ch in password)
has_digit = any(ch.isdigit() for ch in password)

if len(password) >= 8 and has_letter and has_digit:
    print("Password meets the given rules")
else:
    print("Password does not meet the given rules")
```

Test cases and expected output

Test case 1: Input: Python123

Expected output: Password meets the given rules

Test case 2: Input: hello

Expected output: Password does not meet the given rules

Conclusion

This program solves the given problem using basic Python concepts such as variables, conditions, loops, strings, lists, nested lists, or tuples. It displays the required result for the sample data.

8. String-Based Examination Answer Evaluation

Problem statement: Compare a student's answers with an answer key and count correct, incorrect, and unanswered responses.

Objectives

- Compare answer strings.
- Calculate the number of correct, incorrect, and unanswered answers.

Assumptions

- Each answer is one letter.
- A blank answer is unanswered.
- No negative marking is applied.

Algorithm

1. Start.
2. Store the answer key and student answers.
3. Set all counters to zero.
4. Compare each answer with the key.
5. Update correct, incorrect, or unanswered counter.
6. Display all counts.
7. Stop.

Flowchart: Start → Store key and student answers → Compare each answer → Update counters → Display result → Stop

Pseudocode

```
BEGIN
  key = [A, B, C, D, A]
  answers = [A, C, blank, D, A]
  set counters to zero
  FOR each answer position
    IF blank increase unanswered
    ELSE IF equal to key increase correct
    ELSE increase incorrect
  PRINT counters
END
```

Python source code

```
key = ["A", "B", "C", "D", "A"]
answers = ["A", "C", "", "D", "A"]
correct = 0
incorrect = 0
unanswered = 0

for i in range(len(key)):
    if answers[i] == "":
        unanswered += 1
    elif answers[i] == key[i]:
        correct += 1
    else:
        incorrect += 1

print("Correct answers:", correct)
print("Incorrect answers:", incorrect)
print("Unanswered:", unanswered)
```

Test cases and expected output

Test case 1: Input: Predefined sample data

Expected output: Correct answers: 3 | Incorrect answers: 1 | Unanswered: 1

Conclusion

This program solves the given problem using basic Python concepts such as variables, conditions, loops, strings, lists, nested lists, or tuples. It displays the required result for the sample data.

9. Student Marks Matrix Analysis

Problem statement: Use a nested list to store marks of students in different subjects and calculate each student's total and average marks.

Objectives

- Use a nested list to store marks.
- Calculate total and average marks for each student.

Assumptions

- Each student has marks in three subjects.
- Marks are between 0 and 100.
- The example contains three students.

Algorithm

1. Start.
2. Store student names and marks in a nested list.
3. For each student, calculate total marks.
4. Calculate average by dividing total by number of subjects.
5. Display total and average.
6. Stop.

Flowchart: Start → Store student marks in nested list → Calculate total and average → Display each student's result → Stop

Pseudocode

```
BEGIN
  FOR each student and marks row
    total = sum(marks)
    average = total / number of subjects
    PRINT student, total, average
  END FOR
END
```

Python source code

```
students = [
    ("Asha", [80, 75, 90]),
    ("Ravi", [65, 70, 68]),
    ("Sara", [92, 88, 95])
]

for name, marks in students:
    total = sum(marks)
    average = total / len(marks)
    print(name, "total:", total, "average:", round(average, 2))
```

Test cases and expected output

Test case 1: Input: Predefined sample data

Expected output: Asha total: 245 average: 81.67 | Ravi total: 203 average: 67.67 | Sara total: 275 average: 91.67

Conclusion

This program solves the given problem using basic Python concepts such as variables, conditions, loops, strings, lists, nested lists, or tuples. It displays the required result for the sample data.

10. Tuple-Based Product Catalogue Audit

Problem statement: Store product details in tuples and calculate the total value of available stock. Identify products that need replenishment.

Objectives

- Use tuples to store product information.
- Calculate stock value and identify low-stock products.

Assumptions

- Each tuple contains product name, price, and quantity.
- Stock value = price × quantity.
- A quantity below 5 needs replenishment.

Algorithm

1. Start.
2. Store product details as tuples.
3. For each tuple, unpack name, price, and quantity.
4. Calculate stock value.
5. Display stock value.
6. If quantity is below 5, show replenishment warning.
7. Stop.

Flowchart: Start → Store product tuples → Unpack product details → Calculate stock value → Check quantity → Display report → Stop

Pseudocode

```
BEGIN
  FOR each (name, price, quantity) in products
    value = price * quantity
    PRINT name, value
    IF quantity < 5 THEN PRINT replenish
  END FOR
END
```

Python source code

```
products = [
    ("Notebook", 50, 10),
    ("Pen", 10, 3),
    ("Bag", 600, 2)
]

for name, price, quantity in products:
    value = price * quantity
    print(name, "stock value: Rs.", value)
    if quantity < 5:
        print("Replenishment needed")
```

Test cases and expected output

Test case 1: Input: Predefined sample data

Expected output: Notebook stock value: Rs. 500 | Pen stock value: Rs. 30 | Replenishment needed | Bag stock value: Rs. 1200 | Replenishment needed

Conclusion

This program solves the given problem using basic Python concepts such as variables, conditions, loops, strings, lists, nested lists, or tuples. It displays the required result for the sample data.